

Tutorial/Lab 2 – OOP Review 2

Aim

This tutorial/lab aims to review object-oriented programming concepts, such as abstraction, encapsulation, inheritance and polymorphism. `ArrayList` is also reviewed in this session.

Tips

Create a good test set first, which can cover all kinds of cases of input values. This will be helpful to test your code later after you write your solution, but also in understanding the problem statement before you start solving the problem.

Exercises

Exercise 2.1: My 2D Point Implementations

- Design a class named `MyPoint` to represent a point with `x`- and `y`-coordinates.
- The class contains:
 - The data fields (instance variables) `x` and `y` that represent the coordinates with getter methods.
 - An empty constructor that creates a point `(0.0, 0.0)`.
 - A constructor that constructs a point with specified coordinates.
 - A (instance) method named `distance` that returns the distance from this point to another point of the `MyPoint` type.
 - A static method (function) also named `distance` that returns the distance from two `MyPoint` objects.
- Write a test program that creates the three points `(0.0, 0.0)`, `(10.25, 20.8)` and `(13.25, 24.8)` and displays the distance between them using both distance implementations.
Notice the difference between invoking an instance method and a static method (functions)!

Exercise 2.2: Circle2D Class

Define the `Circle2D` class that contains:

- Two `double` data fields named `x` and `y` that specify the center of the circle with getter methods.
- A data field `radius` with a getter method.
- A no-arg constructor that creates a default circle with `(0, 0)` for `(x, y)` and `1` for `radius`.

- A constructor that creates a circle with the specified `x`, `y`, and `radius`.
- A method `getArea()` that returns the area of the circle.
- A method `getPerimeter()` that returns the perimeter of the circle.
- A method `contains(double x, double y)` that returns `true` if the specified point `(x, y)` is inside this circle (see Figure 10.21a).
- A method `contains(Circle2D circle)` that returns `true` if the specified circle is inside this circle (see Figure 10.21b).
- A method `overlaps(Circle2D circle)` that returns `true` if the specified circle overlaps with this circle (see Figure 10.21c).
- **Draw the UML diagram** for the class, then **implement the class**.
- Write a **test program** that creates a `Circle2D` object `c1` (`new Circle2D(2, 2, 5.5)`), displays its area and perimeter, and displays the result of `c1.contains(3, 3)`, `c1.contains(new Circle2D(4, 5, 10.5))`, and `c1.overlaps(new Circle2D(3, 5, 2.3))`.

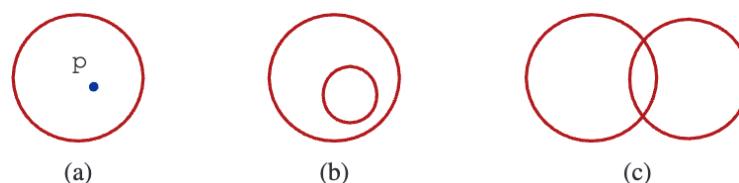


FIGURE 10.21 (a) A point is inside the circle. (b) A circle is inside another circle. (c) A circle overlaps another circle.

Exercise 2.3: Big Integers Divisible by 2 or 3

- Write a function that prints the first 10 numbers with 50 decimal digits that are divisible by 2 or 3.

Exercise 2.4: Triangle Class

Design a class named `Triangle` that extends `GeometricObject` (you can download the class file `GeometricObject.java` from Learning Mall). The class contains:

- Three `double` data fields named `side1`, `side2`, and `side3` with default values `1.0` to denote three sides of a triangle.
- A no-arg constructor that creates a default triangle.
- A constructor that creates a triangle with the specified `side1`, `side2`, and `side3`.
- The accessor methods for all three data fields.
- A method named `getArea()` that returns the area of this triangle.
- A method named `getPerimeter()` that returns the perimeter of this triangle.
- A method named `toString()` that returns a string description for the triangle.

The formula for computing the area of a triangle is:

$$s = (\text{side1} + \text{side2} + \text{side3})/2;$$
$$\text{area} = \sqrt{s(s - \text{side1})(s - \text{side2})(s - \text{side3})}$$

The `toString()` method should be implemented as follows:

```
return "Triangle: side1 = " + side1 + " side2 = " + side2  
+" side3 = " + side3;
```

Write a **test program** that prompts the user to enter three sides of the triangle, a color, and a Boolean value to indicate whether the triangle is filled. The program should create a `Triangle` object with these sides and set the `color` and `filled` properties using the input. The program should display the area, perimeter, color, and true or false to indicate whether it is filled or not.

Exercise 2.5: Person, Student, Employee, Faculty, Staff

- Design a class named `Person` and its two **subclasses** named `Student` and `Employee`.
- Make `Faculty` and `Staff` **subclasses** of `Employee`.
- A person has a name, address, phone number, and e-mail address.
- A student has a class status (freshman, sophomore, junior, or senior). Define the status as a constant.
- An employee has an office, salary, and date hired. Use the `LocalDate` class to create an object for date hired.
- A faculty member has office hours and a rank.
- A staff member has a title.
- Add a constructor that only takes name to each class (you may add other constructors).
- Override the `toString` method in each class to display the class name and the person's name.
- What should the access modifier of name in `Person` be?
- Write a test program that creates a `Person`, `Student`, `Employee`, `Faculty`, and `Staff` objects in an array, and invokes their `toString()` methods using **polymorphism**.

Exercise 2.6: MyStack using Inheritance

- In Lecture 2, we implement `MyStack` using composition.
- Now, implement a new `MyStack` class that extends `ArrayList` Instead.
 - The class definition should be
`public class MyStack extends ArrayList<Object>`
 - The methods are `isEmpty()`, `getSize()`, `peek()`, `pop()`,
`push(Object o)`, `search(Object o)` and `toString()`.
- Write a test program that prompts the user to enter five strings and displays them in reverse order.

Exercise 2.7: Tax Class

- Design a class named `Tax` to contain the following instance data fields:
 - `int filingStatus`: One of the four tax-filing statuses:
 - 0 — single filer
 - 1 — married filing jointly or qualifying widow(er)
 - 2 — married filing separately
 - 3 — head of household.
 - Use the public static constants `SINGLE_FILER (0)`,
`MARRIED_JOINTLY_OR_QUALIFYING_WIDOW_ER (1)`,
`MARRIED_SEPARATELY (2)`, `HEAD_OF_HOUSEHOLD (3)` to represent the statuses.
 - `int[][] brackets`: Stores the tax brackets for each filing status.
 - `double[] rates`: Stores the tax rates for each bracket.
 - `double taxableIncome`: Stores the taxable income.
- Provide the getter and setter methods for each data field.
- Provide a `getTax()` method that returns the tax.
 - Note that each tax rate is applied to a certain amount of taxable income. For example, of a taxable income of \$400,000 for single filers in 2009 (see the 2009 tax rates in Table 3.2 below), \$8,350 is taxed at 10%, (33,950 - 8,350) at 15%, (82,250 - 33,950) at 25%, (171,550 - 82,250) at 28%, (372,950 - 171,550) at 33%, and (400,000 - 372,950) at 35%.
- Provide a no-arg constructor, plus the following constructor:
`Tax(filingStatus, brackets, rates, taxableIncome).`
- Draw the **UML diagram** for the class and then **implement the class**.
- Write a **test program** that uses the `Tax` class to print the 2001 and 2009 tax tables for taxable income from \$50,000 to \$60,000 with intervals of \$1,000 for all four statuses. Refer to the following tables for the 2001 and 2009 tax rates:

TABLE 10.1 2001 U.S. Federal Personal Tax Rates

<i>Tax Rate</i>	<i>Single Filers</i>	<i>Married—Filing Jointly or Qualifying Widow(er)</i>	<i>Married—Filing Separately</i>	<i>Head of Household</i>
15%	Up to \$27,050	Up to \$45,200	Up to \$22,600	Up to \$36,250
27.5%	\$27,051–\$65,550	\$45,201–\$109,250	\$22,601–\$54,625	\$36,251–\$93,650
30.5%	\$65,551–\$136,750	\$109,251–\$166,500	\$54,626–\$83,250	\$93,651–\$151,650
35.5%	\$136,751–\$297,350	\$166,501–\$297,350	\$83,251–\$148,675	\$151,651–\$297,350
39.1%	\$297,351 or more	\$297,351 or more	\$ 148,676 or more	\$297,351 or more

TABLE 3.2 2009 U.S. Federal Personal Tax Rates

<i>Marginal Tax Rate</i>	<i>Single</i>	<i>Married Filing Jointly or Qualifying Widow(er)</i>	<i>Married Filing Separately</i>	<i>Head of Household</i>
10%	\$0–\$8,350	\$0–\$16,700	\$0–\$8,350	\$0–\$11,950
15%	\$8,351–\$33,950	\$16,701–\$67,900	\$8,351–\$33,950	\$11,951–\$45,500
25%	\$33,951–\$82,250	\$67,901–\$137,050	\$33,951–\$68,525	\$45,501–\$117,450
28%	\$82,251–\$171,550	\$137,051–\$208,850	\$68,526–\$104,425	\$117,451–\$190,200
33%	\$171,551–\$372,950	\$208,851–\$372,950	\$104,426–\$186,475	\$190,201–\$372,950
35%	\$372,951+	\$372,951+	\$186,476+	\$372,951+